

El Gato | Tic Tac Toe - Enunciado

// 01

Arquitectura e instancias

01.1

Instancias mínimas para una partida

Requisito de despliegue: Para que dos jugadores puedan jugar entre sí, deben estar activas simultáneamente al menos **3 instancias**: 1 instancia del MS Server y 2 instancias del MS Cliente (una por jugador). Cada instancia del cliente corre en un puerto diferente.

TOPOLOGÍA MÍNIMA DE DESPLIEGUE



Ambos clientes se conectan al mismo server. Cada cliente administra su propio jugador y actúa como proxy hacia el server para las operaciones de juego. Postman interactúa con **cada cliente por separado**: se simula que el Jugador 1 hace sus jugadas via el cliente en :8081 y el Jugador 2 via el cliente en :8082.

Levantar múltiples instancias del cliente

El MS Cliente debe poder ejecutarse en puertos distintos sin modificar el código. Se configura via `application.yml` o argumento de línea de comandos:

```
# Instancia para Jugador 1
mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8081

# Instancia para Jugador 2 (otra terminal)
mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8082
```

Cada instancia tiene su propia base de datos H2 en memoria, independiente de la otra. El nombre de la base de datos debe incluir el puerto para evitar conflictos si se usa H2 en modo archivo.

01.2 Soporte multi-servidor

Un jugador puede jugar en varios servers simultáneamente

El MS Cliente no está atado a un único server. Cada vez que un jugador realiza una operación de juego (crear partida, unirse, mover), **indica el server destino** en el request. Esto permite que un mismo jugador tenga partidas activas en servidores distintos al mismo tiempo.

El cliente almacena localmente un registro de en qué server está cada partida del jugador, lo que permite consultar el estado correcto sin que el jugador deba recordar la URL del server.

MS #1

gato-server :8080

El árbitro del juego. Gestiona el ciclo de vida de las partidas, aplica las reglas y valida cada movimiento. Requiere que los MS Clientes estén registrados para aceptar sus llamadas.

- Crear partidas (NO_TIME o TIMED)
- Unirse a una partida en espera
- Registrar y validar movimientos
- Retornar tablero actualizado tras cada jugada
- Detectar victoria, empate y timeout de turno
- Estadísticas de partidas por jugador
- Registrar MS Clientes autorizados (auto-registro al arranque)

MS #2

gato-client :8081 / :8082

La fachada del jugador. Administra jugadores localmente y actúa como proxy inteligente hacia el server destino. Al arrancar, se registra automáticamente en el server con credenciales de MS.

- Registrar y consultar el jugador propio
- Auto-registro en el server al arranque (`ApplicationRunner`)
- Recibir `serverUrl` en cada operación de juego
- Registrar localmente: `gameld` → `serverUrl`
- Delegar al server correcto según la partida
- Aplicar fallback si el server no responde
- Ver mis partidas activas en todos los servers

01.3 Flujo de comunicación

AUTO-REGISTRO AL ARRANCAR (MS CLIENTE → MS SERVER)

`gato-client` arranca → `ApplicationRunner` → `POST /api/clients/register` →

gato-server → guarda RegisteredClient

H2 server

A partir de aquí, todas las llamadas del cliente al server incluyen `Authorization: Basic` con las credenciales del MS. El server las valida contra la tabla `registered_clients` via Spring Security.

CREAR PARTIDA (JUGADOR 1 VIA CLIENTE :8081)

Postman → :8081

gato-client

→ HTTP + serverUrl

gato-server :8080

→

H2 server

gato-client

→ guarda gameId → serverUrl

H2 client

MOVER (JUGADOR 2 VIA CLIENTE :8082)

Postman → :8082

gato-client

→ busca serverUrl en H2 client

gato-server :8080

En el endpoint `move`, el jugador no necesita volver a indicar la URL del server. El cliente la recupera del registro local usando el `gameId`.

← 00 Intro

02 MS Server →

-->

